Министерство науки и высшего образования Российской Федерации
Калужский филиал
федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ***ИУК «Информатика и управление»***

КАФЕДРА ***ИУК4 «Программное обеспечение ЭВМ, информационные технологии»***

ДОМАШНЯЯ РАБОТА №1

«Взаимодействие СУБД MongoDB и PHP»

ДИСЦИПЛИНА: «Проектирование систем управления базами данных»

Выполнил: студент гр. ИУК4-41М _____ (Сафронов Н.С.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Глебов Ю.С.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Калуга, 2026

Цель: формирование практических навыков работы создание сервера, создающего запросы для базы данных.

Задание

Вариант 6

Новостной сайт.

Листинг программы

docker-compose.yml

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./app/public:/var/www/html
      - ./docker/nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - app

  app:
    build: ./docker/php
    volumes:
      - ./app:/var/www
    working_dir: /var/www
    depends_on:
      - postgres
      - mongo
    environment:
      - APP_ENV=development

  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: news_db
      POSTGRES_USER: news_user
      POSTGRES_PASSWORD: news_password
    ports:
      - "5432:5432"
    volumes:
      - ./docker/postgres/data:/var/lib/postgresql/data
```

```
      - ./app/migrations:/docker-entrypoint-initdb.d

  mongo:
    image: mongo:6
    command: mongod --bind_ip_all
    ports:
      - "27017:27017"
    volumes:
      - ./docker/mongo/data:/data/db
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5

networks:
  default:
    name: php-project
```

app/.env

```
APP_ENV=development
APP_DEBUG=true
APP_URL=http://localhost

PG_HOST=postgres
PG_PORT=5432
PG_DATABASE=news_db
PG_USER=news_user
PG_PASSWORD=news_password

MONGO_HOST=mongo
MONGO_PORT=27017
MONGO_DATABASE=news_db

SESSION_SECRET=your_secret_key_here
```

app/composer.json

```
{
    "name": "news-app/news",
    "type": "project",
    "require": {
        "php": ">=8.2",
        "mongodb/mongodb": "^1.15",
        "vlucas/phpdotenv": "^5.5"
    },
```

```
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        }
    },
    "require-dev": {
        "phpunit/phpunit": "^10.0"
    },
    "config": {
        "platform": {
            "ext-mongodb": "1.17.2"
        }
    }
}
```

docker/php/Dockerfile

```
FROM php:8.2-fpm

RUN apt-get update && apt-get install -y \
    git \
    curl \
    libpng-dev \
    libjpeg62-turbo-dev \
    libfreetype6-dev \
    libxml2-dev \
    libzip-dev \
    zip \
    unzip \
    libpq-dev \
    libonig-dev \
    libbz2-dev \
    libwebp-dev \
    && rm -rf /var/lib/apt/lists/*

RUN docker-php-ext-configure zip \
    && docker-php-ext-configure bcmath \
    && docker-php-ext-install -j$(nproc) \
        pdo \
        pdo_pgsql \
        pgsql \
        mbstring \
        exif \
        pcntl \
        bcmath \
        zip \
```

```
        bz2

RUN pecl install mongodb-1.17.2 \
    && docker-php-ext-enable mongodb

COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

WORKDIR /var/www

# Запуск PHP-FPM
CMD ["php-fpm"]
```

docker/nginx/default.conf

```
server {
    listen 80;
    server_name localhost;
    root /var/www/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        fastcgi_pass app:9000;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

app/public/index.php

```
<?php

declare(strict_types=1);

require_once __DIR__ . '/../vendor/autoload.php';

use Dotenv\Dotenv;
use App\Core\Router;
use App\Config\Database;
```

```
$dotenv = Dotenv::createImmutable(__DIR__ . '/..');
$dotenv->load();

Database::init();

$router = new Router();

$router->get('/', [\App\Controllers\HomeController::class, 'index']);
$router->get('/news', [\App\Controllers\NewsController::class, 'index']);
$router->get('/news/{slug}', [\App\Controllers\NewsController::class, 'show']);
$router->get('/admin', [\App\Controllers\AdminController::class, 'dashboard']);
$router->get('/admin/login', [\App\Controllers\AdminController::class,
'login']);
$router->post('/admin/login', [\App\Controllers\AdminController::class,
'authenticate']);
$router->get('/admin/news', [\App\Controllers\AdminController::class,
'newsList']);
$router->get('/admin/news/create', [\App\Controllers\AdminController::class,
'createNews']);
$router->post('/admin/news/create', [\App\Controllers\AdminController::class,
'storeNews']);
$router->get('/admin/news/edit/{id}', [\App\Controllers\AdminController::class,
'editNews']);
$router->post('/admin/news/update/{id}',
[\App\Controllers\AdminController::class, 'updateNews']);
$router->post('/admin/news/delete/{id}',
[\App\Controllers\AdminController::class, 'deleteNews']);
$router->get('/admin/logout', [\App\Controllers\AdminController::class,
'logout']);

$router->dispatch();
app/migrations/001_initial.sql
-- Пользователи
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) DEFAULT 'author',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Категории
```

```
CREATE TABLE IF NOT EXISTS categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    slug VARCHAR(100) UNIQUE NOT NULL,
    parent_id INTEGER REFERENCES categories(id) ON DELETE SET NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Метаданные новостей
CREATE TABLE IF NOT EXISTS news_meta (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    slug VARCHAR(255) UNIQUE NOT NULL,
    excerpt TEXT,
    category_id INTEGER REFERENCES categories(id) ON DELETE SET NULL,
    author_id INTEGER REFERENCES users(id) ON DELETE SET NULL,
    status VARCHAR(20) DEFAULT 'draft',
    published_at TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Теги
CREATE TABLE IF NOT EXISTS tags (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL,
    slug VARCHAR(50) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Связь новостей и тегов
CREATE TABLE IF NOT EXISTS news_tags (
    news_id INTEGER REFERENCES news_meta(id) ON DELETE CASCADE,
    tag_id INTEGER REFERENCES tags(id) ON DELETE CASCADE,
    PRIMARY KEY (news_id, tag_id)
);

-- Индексы
CREATE INDEX IF NOT EXISTS idx_news_meta_status ON news_meta(status);
CREATE INDEX IF NOT EXISTS idx_news_meta_published_at ON
news_meta(published_at);
CREATE INDEX IF NOT EXISTS idx_news_meta_category_id ON news_meta(category_id);

-- Тестовые данные
```

```
INSERT INTO users (username, email, password_hash, role) VALUES
('admin', 'admin@example.com',
'$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', 'admin'),
('author', 'author@example.com',
'$2y$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', 'author');

INSERT INTO categories (name, slug) VALUES
('Технологии', 'technologies'),
('Бизнес', 'business'),
('Наука', 'science');
app/src/Config/Database.php
<?php

namespace App\Config;

use PDO;
use PDOException;
use MongoDB\Client;

class Database
{
    private static ?PDO $postgres = null;
    private static ?Client $mongo = null;

    public static function init(): void
    {
        self::getPostgres();
        self::getMongo();
    }

    public static function getPostgres(): PDO
    {
        if (self::$postgres === null) {
            $dsn = sprintf(
                'pgsql:host=%s;port=%s;dbname=%s',
                $_ENV['PG_HOST'],
                $_ENV['PG_PORT'],
                $_ENV['PG_DATABASE']
            );

            try {
                self::$postgres = new PDO(
                    $dsn,
                    $_ENV['PG_USER'],
```

```
                    $_ENV['PG_PASSWORD'],
                    [
                        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                        PDO::ATTR_EMULATE_PREPARES => false,
                    ]
                );
            } catch (PDOException $e) {
                die("PostgreSQL connection failed: " . $e->getMessage());
            }
        }

        return self::$postgres;
    }

    public static function getMongo(): Client
    {
        if (self::$mongo === null) {
            $uri = sprintf(
                'mongodb://%s:%s',
                $_ENV['MONGO_HOST'],
                $_ENV['MONGO_PORT']
            );

            try {
                self::$mongo = new Client($uri);
            } catch (\Exception $e) {
                die("MongoDB connection failed: " . $e->getMessage());
            }
        }

        return self::$mongo;
    }

    public static function getMongoDatabase(): \MongoDB\Database
    {
        return self::getMongo()->selectDatabase($_ENV['MONGO_DATABASE']);
    }
}
```

app/src/Core/Router.php

```
<?php

namespace App\Core;
```

```
class Router
{
    private array $routes = [];
    private Request $request;
    private Response $response;

    public function __construct()
    {
        $this->request = new Request();
        $this->response = new Response();
    }

    public function get(string $path, array $handler): void
    {
        $this->routes['GET'][$path] = $handler;
    }

    public function post(string $path, array $handler): void
    {
        $this->routes['POST'][$path] = $handler;
    }

    public function dispatch(): void
    {
        $method = $this->request->getMethod();
        $uri = parse_url($this->request->getUri(), PHP_URL_PATH);

        foreach ($this->routes[$method] ?? [] as $route => $handler) {
            $pattern = preg_replace('/\{([a-zA-Z_]+)\}/', '(?P<$1>[^/]+)',
$route);
            $pattern = '#^' . $pattern . '$#';

            if (preg_match($pattern, $uri, $matches)) {
                $params = array_filter($matches, 'is_string',
ARRAY_FILTER_USE_KEY);
                [$controller, $action] = $handler;
                $instance = new $controller();
                $instance->$action($params);
                return;
            }
        }

        http_response_code(404);
        echo '404 - Not Found';
```

```
    }
}
app/src/Core/Request.php
<?php

namespace App\Core;

class Request
{
    public function getMethod(): string
    {
        return $_SERVER['REQUEST_METHOD'];
    }

    public function getUri(): string
    {
        return $_SERVER['REQUEST_URI'];
    }

    public function getPost(string $key, $default = null)
    {
        return $_POST[$key] ?? $default;
    }

    public function getAllPost(): array
    {
        return $_POST;
    }

    public function isAjax(): bool
    {
        return !empty($_SERVER['HTTP_X_REQUESTED_WITH']) &&
               strtolower($_SERVER['HTTP_X_REQUESTED_WITH']) ==
'xmlhttprequest';
    }
}
app/src/Core/Response.php
<?php

namespace App\Core;

class Response
{
    public function redirect(string $url): void
```

```
    {
        header("Location: $url");
        exit;
    }

    public function json(array $data): void
    {
        header('Content-Type: application/json');
        echo json_encode($data);
        exit;
    }

    public function view(string $template, array $data = []): void
    {
        extract($data);
        include __DIR__ . "/../Views/$template";
    }
}
```

app/src/Models/News.php

```
<?php

namespace App\Models;

class News
{
    public int $id;
    public string $title;
    public string $slug;

    public ?string $excerpt;
    public ?string $content_html;
    public ?string $content_json;

    public int $category_id;
    public int $author_id;

    public string $status;
    public ?string $published_at;
    public ?string $created_at;
    public ?string $updated_at;

    public ?string $category_name;
    public ?string $category_slug;
    public ?string $author_name;
```

```
    private array $data = [];

    public function __construct(array $data = [])
    {
        $this->data = $data;

        foreach ($data as $key => $value) {
            if (property_exists($this, $key)) {
                $this->$key = $value;
            }
        }
    }

    public function __get(string $name)
    {
        return $this->data[$name] ?? null;
    }

    public function __isset(string $name): bool
    {
        return isset($this->data[$name]);
    }

    public function isPublished(): bool
    {
        return $this->status === 'published';
    }
}
```

app/src/Models/User.php

```
<?php

namespace App\Models;

class User
{
    public int $id;
    public string $username;
    public string $email;
    public string $password_hash;
    public string $role;
    public ?string $created_at;
    public ?string $updated_at;
```

```
    public function __construct(array $data = [])
    {
        foreach ($data as $key => $value) {
            if (property_exists($this, $key)) {
                $this->$key = $value;
            }
        }
    }

    public function isAdmin(): bool
    {
        return $this->role === 'admin';
    }

    public function isAuthor(): bool
    {
        return in_array($this->role, ['admin', 'author']);
    }
}
```

app/src/Models/Category.php

```
<?php

namespace App\Models;

class Category
{
    public int $id;
    public string $name;
    public string $slug;
    public ?int $parent_id;
    public ?string $created_at;

    public function __construct(array $data = [])
    {
        foreach ($data as $key => $value) {
            if (property_exists($this, $key)) {
                $this->$key = $value;
            }
        }
    }
}
```

app/src/Repositories/Postgres/NewsMetaRepository.php

```
<?php
```

```php
namespace App\Repositories\Postgres;

use App\Config\Database;
use App\Models\News;
use PDO;

class NewsMetaRepository
{
    private PDO $db;

    public function __construct()
    {
        $this->db = Database::getPostgres();
    }

    public function findAll(int $limit = 10, int $offset = 0): array
    {
        $sql = "SELECT nm.*, c.name as category_name, c.slug as category_slug,
                       u.username as author_name
                FROM news_meta nm
                LEFT JOIN categories c ON nm.category_id = c.id
                LEFT JOIN users u ON nm.author_id = u.id
                WHERE nm.status = 'published'
                ORDER BY nm.published_at DESC
                LIMIT :limit OFFSET :offset";

        $stmt = $this->db->prepare($sql);
        $stmt->bindValue(':limit', $limit, PDO::PARAM_INT);
        $stmt->bindValue(':offset', $offset, PDO::PARAM_INT);
        $stmt->execute();

        return $stmt->fetchAll();
    }

    public function findBySlug(string $slug): ?array
    {
        $sql = "SELECT nm.*, c.name as category_name, c.slug as category_slug,
                       u.username as author_name, u.id as author_id
                FROM news_meta nm
                LEFT JOIN categories c ON nm.category_id = c.id
                LEFT JOIN users u ON nm.author_id = u.id
                WHERE nm.slug = :slug";

        $stmt = $this->db->prepare($sql);
```

```
        $stmt->execute(['slug' => $slug]);

        $result = $stmt->fetch();
        return $result ?: null;
    }

    public function findById(int $id): ?array
    {
        $sql = "SELECT * FROM news_meta WHERE id = :id";
        $stmt = $this->db->prepare($sql);
        $stmt->execute(['id' => $id]);

        $result = $stmt->fetch();
        return $result ?: null;
    }

    public function findAllForAdmin(): array
    {
        $sql = "SELECT nm.*, c.name as category_name, u.username as author_name
                FROM news_meta nm
                LEFT JOIN categories c ON nm.category_id = c.id
                LEFT JOIN users u ON nm.author_id = u.id
                ORDER BY nm.created_at DESC";

        $stmt = $this->db->query($sql);
        return $stmt->fetchAll();
    }

    public function create(array $data): int
    {
        $sql = "INSERT INTO news_meta (title, slug, excerpt, category_id,
author_id, status, published_at)
                VALUES (:title, :slug, :excerpt, :category_id, :author_id,
:status, :published_at)
                RETURNING id";

        $stmt = $this->db->prepare($sql);
        $stmt->execute($data);

        return (int) $stmt->fetchColumn();
    }

    public function update(int $id, array $data): bool
    {
```

```
        $sets = [];
        foreach (array_keys($data) as $key) {
            $sets[] = "$key = :$key";
        }
        $sets[] = "updated_at = CURRENT_TIMESTAMP";

        $sql = "UPDATE news_meta SET " . implode(', ', $sets) . " WHERE id =
:id";
        $data['id'] = $id;

        $stmt = $this->db->prepare($sql);
        return $stmt->execute($data);
    }

    public function delete(int $id): bool
    {
        $sql = "DELETE FROM news_meta WHERE id = :id";
        $stmt = $this->db->prepare($sql);
        return $stmt->execute(['id' => $id]);
    }

    public function count(): int
    {
        $sql = "SELECT COUNT(*) FROM news_meta WHERE status = 'published'";
        return (int) $this->db->query($sql)->fetchColumn();
    }
}
app/src/Repositories/Postgres/UserRepository.php
<?php

namespace App\Repositories\Postgres;

use App\Config\Database;
use PDO;

class UserRepository
{
    private PDO $db;

    public function __construct()
    {
        $this->db = Database::getPostgres();
    }
```

```
    public function findByEmail(string $email): ?array
    {
        $sql = "SELECT * FROM users WHERE email = :email";
        $stmt = $this->db->prepare($sql);
        $stmt->execute(['email' => $email]);
        $result = $stmt->fetch();
        return $result ?: null;
    }

    public function findById(int $id): ?array
    {
        $sql = "SELECT * FROM users WHERE id = :id";
        $stmt = $this->db->prepare($sql);
        $stmt->execute(['id' => $id]);
        $result = $stmt->fetch();
        return $result ?: null;
    }

    public function create(string $username, string $email, string
$passwordHash, string $role = 'author'): int
    {
        $sql = "INSERT INTO users (username, email, password_hash, role)
                VALUES (:username, :email, :password_hash, :role) RETURNING id";
        $stmt = $this->db->prepare($sql);
        $stmt->execute([
            'username' => $username,
            'email' => $email,
            'password_hash' => $passwordHash,
            'role' => $role
        ]);
        return (int) $stmt->fetchColumn();
    }
}
app/src/Repositories/Postgres/CategoryRepository.php
<?php

namespace App\Repositories\Postgres;

use App\Config\Database;
use App\Models\Category;
use PDO;

class CategoryRepository
{
```

```
    private PDO $db;

    public function __construct()
    {
        $this->db = Database::getPostgres();
    }

    public function findAll(): array
    {
        $sql = "SELECT * FROM categories ORDER BY name";
        $stmt = $this->db->query($sql);
        return $stmt->fetchAll();
    }

    public function findById(int $id): ?array
    {
        $sql = "SELECT * FROM categories WHERE id = :id";
        $stmt = $this->db->prepare($sql);
        $stmt->execute(['id' => $id]);
        $result = $stmt->fetch();
        return $result ?: null;
    }

    public function create(string $name, string $slug, ?int $parentId = null):
int
    {
        $sql = "INSERT INTO categories (name, slug, parent_id) VALUES (:name,
:slug, :parent_id) RETURNING id";
        $stmt = $this->db->prepare($sql);
        $stmt->execute([
            'name' => $name,
            'slug' => $slug,
            'parent_id' => $parentId
        ]);
        return (int) $stmt->fetchColumn();
    }
}
app/src/Repositories/Mongo/NewsContentRepository.php
<?php

namespace App\Repositories\Mongo;

use App\Config\Database as AppConfigDatabase;
use MongoDB\Client;
```

```
use MongoDB\Database as MongoDatabase;
use MongoDB\BSON\UTCDateTime;

class NewsContentRepository
{
    private MongoDatabase $db;
    private \MongoDB\Collection $collection;

    public function __construct()
    {
        $this->db = AppConfigDatabase::getMongoDatabase();
        $this->collection = $this->db->selectCollection('news_content');
    }

    public function findByNewsId(int $newsId): ?array
    {
        $result = $this->collection->findOne(
            ['news_meta_id' => $newsId],
            ['sort' => ['version' => -1]]
        );

        return $result !== null ? (array) $result : null;
    }

    public function create(int $newsId, string $contentHtml, string
$contentJson, int $authorId): void
    {
        $this->collection->insertOne([
            'news_meta_id' => $newsId,
            'version' => 1,
            'content_html' => $contentHtml,
            'content_json' => $contentJson,
            'author_id' => $authorId,
            'created_at' => new UTCDateTime()
        ]);
    }

    public function update(int $newsId, string $contentHtml, string
$contentJson, int $authorId): void
    {
        $latest = $this->findByNewsId($newsId);
        $version = $latest ? ($latest['version'] + 1) : 1;

        $this->collection->insertOne([
```

```
            'news_meta_id' => $newsId,
            'version' => $version,
            'content_html' => $contentHtml,
            'content_json' => $contentJson,
            'author_id' => $authorId,
            'created_at' => new UTCDateTime()
        ]);
    }

    public function deleteByNewsId(int $newsId): void
    {
        $this->collection->deleteMany(['news_meta_id' => $newsId]);
    }

    public function getVersions(int $newsId): array
    {
        $cursor = $this->collection->find(
            ['news_meta_id' => $newsId],
            ['sort' => ['version' => -1]]
        );

        return array_map(fn($doc) => (array) $doc, iterator_to_array($cursor));
    }
}
```

app/src/Services/NewsService.php

```
<?php

namespace App\Services;

use App\Models\News;
use App\Repositories\Postgres\NewsMetaRepository;
use App\Repositories\Postgres\CategoryRepository;
use App\Repositories\Mongo\NewsContentRepository;

class NewsService
{
    private NewsMetaRepository $metaRepo;
    private CategoryRepository $categoryRepo;
    private NewsContentRepository $contentRepo;

    public function __construct(
        NewsMetaRepository $metaRepo,
        CategoryRepository $categoryRepo,
        NewsContentRepository $contentRepo
```

```
) {
    $this->metaRepo = $metaRepo;
    $this->categoryRepo = $categoryRepo;
    $this->contentRepo = $contentRepo;
}

public function getPublishedNews(int $limit = 10, int $offset = 0): array
{
    $newsList = $this->metaRepo->findAll($limit, $offset);

    return array_map(function($news) {
        $model = new News($news);
        $content = $this->contentRepo->findByNewsId($model->id);
        if ($content) {
            $model->content_html = $content['content_html'] ?? '';
            $model->content_json = $content['content_json'] ?? '{}';
        }
        return $model;
    }, $newsList);
}

public function getNewsBySlug(string $slug): ?News
{
    $meta = $this->metaRepo->findBySlug($slug);
    if (!$meta) {
        return null;
    }

    $news = new News($meta);
    $content = $this->contentRepo->findByNewsId($news->id);

    if ($content) {
        $news->content_html = $content['content_html'] ?? '';
        $news->content_json = $content['content_json'] ?? '{}';
    }

    return $news;
}

public function createNews(array $data, int $authorId): int
{
    $metaData = [
        'title' => $data['title'],
        'slug' => $this->generateSlug($data['title']),
```

```
            'excerpt' => $data['excerpt'] ?? null,
            'category_id' => $data['category_id'] ?? null,
            'author_id' => $authorId,
            'status' => $data['status'] ?? 'draft',
            'published_at' => ($data['status'] === 'published') ? date('Y-m-d
H:i:s') : null
        ];

        $newsId = $this->metaRepo->create($metaData);

        $this->contentRepo->create(
            $newsId,
            $data['content_html'] ?? '',
            $data['content_json'] ?? '{}',
            $authorId
        );

        return $newsId;
    }

    public function updateNews(int $id, array $data, int $authorId): bool
    {
        $metaData = [
            'title' => $data['title'],
            'slug' => $this->generateSlug($data['title']),
            'excerpt' => $data['excerpt'] ?? null,
            'category_id' => $data['category_id'] ?? null,
            'status' => $data['status'] ?? 'draft',
        ];

        if ($data['status'] === 'published') {
            $metaData['published_at'] = date('Y-m-d H:i:s');
        }

        $this->metaRepo->update($id, $metaData);

        $this->contentRepo->update(
            $id,
            $data['content_html'] ?? '',
            $data['content_json'] ?? '{}',
            $authorId
        );

        return true;
```

```
}

public function deleteNews(int $id): bool
{
    $this->contentRepo->deleteByNewsId($id);
    return $this->metaRepo->delete($id);
}

public function getAllNewsForAdmin(): array
{
    return $this->metaRepo->findAllForAdmin();
}

public function getCategories(): array
{
    return $this->categoryRepo->findAll();
}

private function generateSlug(string $title): string
{
    $slug = $this->transliterate($title);
    $slug = strtolower($slug);
    $slug = preg_replace('/[^a-z0-9_-]/', '-', $slug);
    $slug = preg_replace('/-+/', '-', $slug);
    $slug = trim($slug, '-');
    $baseSlug = $slug;
    $counter = 1;

    while ($this->metaRepo->findBySlug($slug)) {
        $slug = $baseSlug . '-' . $counter;
        $counter++;
    }

    return $slug;
}

private function transliterate(string $text): string
{
    $converter = [
        'а' => 'a',    'б' => 'b',    'в' => 'v',    'г' => 'g',
        'д' => 'd',    'е' => 'e',    'ё' => 'yo',   'ж' => 'zh',
        'з' => 'z',    'и' => 'i',    'й' => 'y',    'к' => 'k',
        'л' => 'l',    'м' => 'm',    'н' => 'n',    'о' => 'o',
        'п' => 'p',    'р' => 'r',    'с' => 's',    'т' => 't',
```

```
            'у' => 'u',    'ф' => 'f',    'х' => 'h',    'ц' => 'ts',
            'ч' => 'ch',   'ш' => 'sh',   'щ' => 'sch',  'ъ' => '',
            'ы' => 'y',    'ь' => '',     'э' => 'e',    'ю' => 'yu',
            'я' => 'ya',

            'А' => 'A',    'Б' => 'B',    'В' => 'V',    'Г' => 'G',
            'Д' => 'D',    'Е' => 'E',    'Ё' => 'Yo',   'Ж' => 'Zh',
            'З' => 'Z',    'И' => 'I',    'Й' => 'Y',    'К' => 'K',
            'Л' => 'L',    'М' => 'M',    'Н' => 'N',    'О' => 'O',
            'П' => 'P',    'Р' => 'R',    'С' => 'S',    'Т' => 'T',
            'У' => 'U',    'Ф' => 'F',    'Х' => 'H',    'Ц' => 'Ts',
            'Ч' => 'Ch',   'Ш' => 'Sh',   'Щ' => 'Sch',  'Ъ' => '',
            'Ы' => 'Y',    'Ь' => '',     'Э' => 'E',    'Ю' => 'Yu',
            'Я' => 'Ya',

            '№' => '',     '&' => 'and',  '@' => 'at',
        ];

        return strtr($text, $converter);
    }
}
```

app/src/Controllers/HomeController.php

```
<?php

namespace App\Controllers;

use App\Core\Response;
use App\Services\NewsService;
use App\Repositories\Postgres\NewsMetaRepository;
use App\Repositories\Postgres\CategoryRepository;
use App\Repositories\Mongo\NewsContentRepository;

class HomeController
{
    private Response $response;
    private NewsService $newsService;

    public function __construct()
    {
        $this->response = new Response();
        $this->newsService = new NewsService(
            new NewsMetaRepository(),
            new CategoryRepository(),
            new NewsContentRepository()
```

```
        );
    }

    public function index(): void
    {
        $news = $this->newsService->getPublishedNews(5);
        $categories = $this->newsService->getCategories();

        $this->response->view('layouts/main.php', [
            'title' => 'Главная',
            'content' => 'home/index.php',
            'news' => $news,
            'categories' => $categories
        ]);
    }
}
```

app/src/Controllers/NewsController.php

```
<?php

namespace App\Controllers;

use App\Core\Response;
use App\Services\NewsService;
use App\Repositories\Postgres\NewsMetaRepository;
use App\Repositories\Postgres\CategoryRepository;
use App\Repositories\Mongo\NewsContentRepository;

class NewsController
{
    private Response $response;
    private NewsService $newsService;

    public function __construct()
    {
        $this->response = new Response();
        $this->newsService = new NewsService(
            new NewsMetaRepository(),
            new CategoryRepository(),
            new NewsContentRepository()
        );
    }

    public function index(): void
    {
```

```
        $page = max(1, (int)($_GET['page'] ?? 1));
        $limit = 10;
        $offset = ($page - 1) * $limit;

        $news = $this->newsService->getPublishedNews($limit, $offset);
        $categories = $this->newsService->getCategories();

        $totalNews = count($this->newsService->getAllNewsForAdmin());
        $totalPages = ceil($totalNews / $limit);

        $this->response->view('layouts/main.php', [
            'title' => 'Новости',
            'content' => 'news/index.php',
            'news' => $news,
            'categories' => $categories,
            'currentPage' => $page,
            'totalPages' => $totalPages
        ]);
    }

    public function show(array $params): void
    {
        $news = $this->newsService->getNewsBySlug($params['slug']);

        if (!$news) {
            http_response_code(404);
            $this->response->view('layouts/main.php', [
                'title' => 'Страница не найдена',
                'content' => 'errors/404.php',
                'categories' => $this->newsService->getCategories()
            ]);
            return;
        }

        $categories = $this->newsService->getCategories();

        $this->response->view('layouts/main.php', [
            'title' => $news->title ?? 'Новость',
            'content' => 'news/show.php',
            'news' => $news,
            'categories' => $categories
        ]);
}
}
```

app/src/Controllers/AdminController.php

```php
<?php

namespace App\Controllers;

use App\Core\Response;
use App\Services\NewsService;
use App\Repositories\Postgres\NewsMetaRepository;
use App\Repositories\Postgres\CategoryRepository;
use App\Repositories\Postgres\UserRepository;
use App\Repositories\Mongo\NewsContentRepository;

class AdminController
{
    private Response $response;
    private NewsService $newsService;
    private UserRepository $userRepo;

    public function __construct()
    {
        $this->response = new Response();
        $this->newsService = new NewsService(
            new NewsMetaRepository(),
            new CategoryRepository(),
            new NewsContentRepository()
        );
        $this->userRepo = new UserRepository();
    }

    private function requireAuth(): ?array
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        if (!isset($_SESSION['user_id'])) {
            $this->response->redirect('/admin/login');
            return null;
        }

        $user = $this->userRepo->findById($_SESSION['user_id']);
        if (!$user || !in_array($user['role'], ['admin', 'author'])) {
            session_destroy();
            $this->response->redirect('/admin/login');
```

```
            return null;
        }

        return $user;
    }

    public function login(): void
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        $this->response->view('layouts/main.php', [
            'title' => 'Вход',
            'content' => 'admin/login.php',
            'error' => null
        ]);
    }

    public function authenticate(): void
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        $email = $_POST['email'] ?? '';
        $password = $_POST['password'] ?? '';

        $user = $this->userRepo->findByEmail($email);

        if ($user && password_verify($password, $user['password_hash'])) {
            $_SESSION['user_id'] = $user['id'];
            $_SESSION['username'] = $user['username'];
            $_SESSION['role'] = $user['role'];
            $this->response->redirect('/admin');
        } else {
            $this->response->view('layouts/main.php', [
                'title' => 'Вход',
                'content' => 'admin/login.php',
                'error' => 'Неверный email или пароль'
            ]);
        }
    }
```

```
public function logout(): void
{
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }
    session_destroy();
    $this->response->redirect('/admin/login');
}

public function dashboard(): void
{
    $user = $this->requireAuth();
    if (!$user) return;

    $news = $this->newsService->getAllNewsForAdmin();
    $categories = $this->newsService->getCategories();

    $this->response->view('layouts/main.php', [
        'title' => 'Админ-панель',
        'content' => 'admin/dashboard.php',
        'news' => $news,
        'categories' => $categories,
        'currentUser' => $user
    ]);
}

public function newsList(): void
{
    $this->dashboard();
}

public function createNews(): void
{
    $user = $this->requireAuth();
    if (!$user) return;

    $categories = $this->newsService->getCategories();

    $this->response->view('layouts/main.php', [
        'title' => 'Создать новость',
        'content' => 'admin/news-form.php',
        'categories' => $categories,
        'news' => null,
        'currentUser' => $user
```

```
        ]);
    }

    public function storeNews(): void
    {
        $user = $this->requireAuth();
        if (!$user) return;

        $data = [
            'title' => $_POST['title'] ?? '',
            'excerpt' => $_POST['excerpt'] ?? '',
            'content_html' => $_POST['content_html'] ?? '',
            'content_json' => $_POST['content_json'] ?? '{}',
            'category_id' => $_POST['category_id'] ?? null,
            'status' => $_POST['status'] ?? 'draft'
        ];

        try {
            $this->newsService->createNews($data, $user['id']);
            $this->response->redirect('/admin');
        } catch (\Exception $e) {
            $this->response->view('layouts/main.php', [
                'title' => 'Ошибка',
                'content' => 'admin/news-form.php',
                'error' => $e->getMessage(),
                'categories' => $this->newsService->getCategories(),
                'news' => null,
                'currentUser' => $user
            ]);
        }
    }

    public function editNews(array $params): void
    {
        $user = $this->requireAuth();
        if (!$user) return;

        $newsMeta = (new NewsMetaRepository())->findById($params['id']);
        if (!$newsMeta) {
            $this->response->redirect('/admin');
            return;
        }

        $newsContent = (new NewsContentRepository())-
```

```
>findByNewsId($params['id']);

        $news = (object) array_merge($newsMeta, [
            'content_html' => $newsContent['content_html'] ?? '',
            'content_json' => $newsContent['content_json'] ?? '{}'
        ]);

        $categories = $this->newsService->getCategories();

        $this->response->view('layouts/main.php', [
            'title' => 'Редактировать новость',
            'content' => 'admin/news-form.php',
            'categories' => $categories,
            'news' => $news,
            'currentUser' => $user
        ]);
    }

    public function updateNews(array $params): void
    {
        $user = $this->requireAuth();
        if (!$user) return;

        $data = [
            'title' => $_POST['title'] ?? '',
            'excerpt' => $_POST['excerpt'] ?? '',
            'content_html' => $_POST['content_html'] ?? '',
            'content_json' => $_POST['content_json'] ?? '{}',
            'category_id' => $_POST['category_id'] ?? null,
            'status' => $_POST['status'] ?? 'draft'
        ];

        try {
            $this->newsService->updateNews($params['id'], $data, $user['id']);
            $this->response->redirect('/admin');
        } catch (\Exception $e) {
            $this->response->redirect('/admin/news/edit/' . $params['id']);
        }
    }

    public function deleteNews(array $params): void
    {
        $user = $this->requireAuth();
        if (!$user) return;
```

```
        try {
            $this->newsService->deleteNews($params['id']);
            $this->response->redirect('/admin');
        } catch (\Exception $e) {
            $this->response->redirect('/admin');
        }
    }
}
```

app/src/Views/layouts/main.php

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title><?= htmlspecialchars($title ?? 'Новости') ?> - News App</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; line-height: 1.6; background:
#f4f4f4; }
        .container { max-width: 1200px; margin: 0 auto; padding: 20px; }
        header { background: #333; color: #fff; padding: 1rem 0; }
        header nav { display: flex; justify-content: space-between; align-items:
center; }
        header nav a { color: #fff; text-decoration: none; margin-left: 20px; }
        header nav a:hover { text-decoration: underline; }
        .content { background: #fff; padding: 20px; margin-top: 20px; border-
radius: 5px; }
        .news-item { border-bottom: 1px solid #ddd; padding: 20px 0; }
        .news-item h2 a { color: #333; text-decoration: none; }
        .news-item h2 a:hover { color: #007bff; }
        .news-meta { color: #666; font-size: 0.9em; margin: 10px 0; }
        .btn { display: inline-block; padding: 10px 20px; background: #007bff;
color: #fff; text-decoration: none; border-radius: 5px; }
        .btn:hover { background: #0056b3; }
        .btn-danger { background: #dc3545; }
        .btn-danger:hover { background: #c82333; }
        .form-group { margin-bottom: 15px; }
        .form-group label { display: block; margin-bottom: 5px; font-weight:
bold; }
        .form-group input, .form-group textarea, .form-group select { width:
100%; padding: 10px; border: 1px solid #ddd; border-radius: 5px; }
        .form-group textarea { min-height: 300px; }
        table { width: 100%; border-collapse: collapse; }
```

```
        table th, table td { padding: 10px; text-align: left; border-bottom: 1px
solid #ddd; }
        table th { background: #f8f9fa; }
        .status-published { color: green; }
        .status-draft { color: orange; }
        .pagination { margin: 20px 0; text-align: center; }
        .pagination a { margin: 0 5px; padding: 5px 10px; background: #007bff;
color: #fff; text-decoration: none; border-radius: 3px; }
        .error { background: #f8d7da; color: #721c24; padding: 10px; border-
radius: 5px; margin-bottom: 20px; }
        footer { text-align: center; padding: 20px; margin-top: 20px; color:
#666; }
    </style>
</head>
<body>
    <header>
        <div class="container">
            <nav>
                <div>
                    <a href="/">🏠 Главная</a>
                    <a href="/news">📰 Новости</a>
                </div>
                <div>
                    <?php if (isset($_SESSION['user_id'])): ?>
                        <a href="/admin">⚙️ Админка</a>
                        <a href="/admin/logout">🚪 Выход</a>
                        <span>(<?= htmlspecialchars($_SESSION['username'])
?>)</span>
                    <?php else: ?>
                        <a href="/admin/login">🔐 Вход</a>
                    <?php endif; ?>
                </div>
            </nav>
        </div>
    </header>

    <div class="container">
        <?php if (isset($error)): ?>
            <div class="error"><?= htmlspecialchars($error) ?></div>
        <?php endif; ?>

        <div class="content">
            <?php include __DIR__ . '/../' . $content; ?>
```

```
        </div>
    </div>

    <footer>
        <p>&copy; <?= date('Y') ?> News App. PostgreSQL + MongoDB</p>
    </footer>
</body>
</html>
app/src/Views/home/index.php
<h1>Последние новости</h1>

<?php if (empty($news)): ?>
    <p>Новостей пока нет.</p>
<?php else: ?>
    <?php foreach ($news as $item): ?>
        <div class="news-item">
            <h2>
                <a href="/news/<?= htmlspecialchars($item->slug ?? '') ?>">
                    <?= htmlspecialchars($item->title ?? 'Без названия') ?>
                </a>
            </h2>
            <div class="news-meta">
                <?php if (!empty($item->category_name)): ?>
                    <span>📁 <?= htmlspecialchars($item->category_name)
?></span> |
                <?php endif; ?>
                <span>✍️ <?= htmlspecialchars($item->author_name ?? 'Аноним')
?></span> |
                <span>🗓️ <?= !empty($item->published_at) ? date('d.m.Y',
strtotime($item->published_at)) : 'Дата не указана' ?></span>
            </div>
            <?php if (!empty($item->excerpt)): ?>
                <p><?= htmlspecialchars(mb_substr($item->excerpt, 0, 200))
?>...</p>
            <?php endif; ?>
            <a href="/news/<?= htmlspecialchars($item->slug ?? '') ?>"
class="btn">Читать далее →</a>
        </div>
    <?php endforeach; ?>
<?php endif; ?>

<div style="margin-top: 20px;">
    <a href="/news" class="btn">Все новости</a>
</div>
```

app/src/Views/news/index.php

```
<h1>Все новости</h1>

<?php if (empty($news)): ?>
    <p>Новостей пока нет.</p>
<?php else: ?>
    <?php foreach ($news as $item): ?>
        <div class="news-item">
            <h2>
                <a href="/news/<?= htmlspecialchars($item->slug ?? '') ?>">
                    <?= htmlspecialchars($item->title ?? 'Без названия') ?>
                </a>
            </h2>
            <div class="news-meta">
                <?php if (!empty($item->category_name)): ?>
                    <span>📁 <?= htmlspecialchars($item->category_name)
?></span> |
                <?php endif; ?>
                <span>✍️ <?= htmlspecialchars($item->author_name ?? 'Аноним')
?></span> |
                <span>🗓️ <?= !empty($item->published_at) ? date('d.m.Y',
strtotime($item->published_at)) : 'Дата не указана' ?></span>
            </div>
            <?php if (!empty($item->excerpt)): ?>
                <p><?= htmlspecialchars(mb_substr($item->excerpt, 0, 200))
?>...</p>
            <?php endif; ?>
            <a href="/news/<?= htmlspecialchars($item->slug ?? '') ?>"
class="btn">Читать далее →</a>
        </div>
    <?php endforeach; ?>
<?php endif; ?>

<?php if (!empty($totalPages) && $totalPages > 1): ?>
    <div class="pagination">
        <?php for ($i = 1; $i <= $totalPages; $i++): ?>
            <a href="/news?page=<?= $i ?>" <?= $i === ($currentPage ?? 1) ?
'style="background:#0056b3;"' : '' ?>><?= $i ?></a>
        <?php endfor; ?>
    </div>
<?php endif; ?>
```

app/src/Views/news/show.php

```
<?php if (!$news): ?>
    <div class="error-message" style="text-align: center; padding: 50px;">
```

```
        <h1>🚫 Новость не найдена</h1>
        <p>К сожалению, запрашиваемая страница не существует.</p>
        <a href="/news" class="btn" style="margin-top: 20px;">← Вернуться к
списку новостей</a>
    </div>
<?php else: ?>
    <article>
        <h1><?= htmlspecialchars($news->title ?? 'Без названия') ?></h1>

        <div class="news-meta" style="margin: 20px 0; padding: 10px; background:
#f8f9fa; border-radius: 5px;">
            <?php if (!empty($news->category_name)): ?>
                <span>📁 <?= htmlspecialchars($news->category_name) ?></span> |
            <?php endif; ?>
            <span>✍️ <?= htmlspecialchars($news->author_name ?? 'Аноним')
?></span> |
            <span>🗓️ <?= !empty($news->published_at) ? date('d.m.Y H:i',
strtotime($news->published_at)) : 'Дата не указана' ?></span>
        </div>

        <div class="news-content" style="margin: 20px 0;">
            <?= $news->content_html ?? '' ?>
        </div>

        <div style="margin-top: 30px; padding-top: 20px; border-top: 1px solid
#ddd;">
            <a href="/news" class="btn">← Назад к новостям</a>
        </div>
    </article>
<?php endif; ?>
app/src/Views/admin/login.php
<h1>Вход в админ-панель</h1>

<form method="POST" action="/admin/login" style="max-width: 400px; margin: 20px
auto;">
    <div class="form-group">
        <label for="email">Email:</label>
        <input type="email" name="email" id="email" required>
    </div>

    <div class="form-group">
        <label for="password">Пароль:</label>
        <input type="password" name="password" id="password" required>
```

```
    </div>

    <button type="submit" class="btn">Войти</button>
</form>

<div style="margin-top: 20px; padding: 15px; background: #e7f3ff; border-radius:
5px;">
    <strong>Тестовые данные:</strong><br>
    Email: admin@example.com<br>
    Пароль: password
</div>
app/src/Views/admin/dashboard.php
<h1>Админ-панель</h1>
<p>Добро пожаловать, <?= htmlspecialchars($currentUser['username']) ?>!</p>

<div style="margin: 20px 0;">
    <a href="/admin/news/create" class="btn">➕ Создать новость</a>
</div>

<h2>Все новости</h2>

<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>Заголовок</th>
            <th>Категория</th>
            <th>Автор</th>
            <th>Статус</th>
            <th>Дата</th>
            <th>Действия</th>
        </tr>
    </thead>
    <tbody>
        <?php if (empty($news)): ?>
            <tr><td colspan="7">Новостей нет</td></tr>
        <?php else: ?>
            <?php foreach ($news as $item): ?>
                <tr>
                    <td><?= $item['id'] ?></td>
                    <td><?= htmlspecialchars($item['title']) ?></td>
                    <td><?= htmlspecialchars($item['category_name'] ?? '-')
?></td>
                    <td><?= htmlspecialchars($item['author_name'] ?? '-')
```

```
?></td>
                    <td class="status-<?= $item['status'] ?>"><?=
htmlspecialchars($item['status']) ?></td>
                    <td><?= date('d.m.Y', strtotime($item['created_at']))
?></td>
                    <td>
                        <a href="/admin/news/edit/<?= $item['id'] ?>"
class="btn" style="padding: 5px 10px;">🖍️</a>
                        <form method="POST" action="/admin/news/delete/<?=
$item['id'] ?>" style="display:inline;" onsubmit="return confirm('Удалить
новость?');">
                            <button type="submit" class="btn btn-danger"
style="padding: 5px 10px;">🗑️</button>
                        </form>
                    </td>
                </tr>
            <?php endforeach; ?>
        <?php endif; ?>
    </tbody>
</table>
app/src/Views/admin/news-form.php
<h1><?= isset($news->id) ? 'Редактировать новость' : 'Создать новость' ?></h1>

<form method="POST" action="<?= isset($news->id) ? '/admin/news/update/' .
$news->id : '/admin/news/create' ?>" style="max-width: 800px;">
    <div class="form-group">
        <label for="title">Заголовок *</label>
        <input type="text" name="title" id="title" value="<?=
htmlspecialchars($news->title ?? '') ?>" required>
    </div>

    <div class="form-group">
        <label for="excerpt">Краткое описание</label>
        <textarea name="excerpt" id="excerpt" rows="3"><?=
htmlspecialchars($news->excerpt ?? '') ?></textarea>
    </div>

    <div class="form-group">
        <label for="category_id">Категория</label>
        <select name="category_id" id="category_id">
            <option value="">-- Выберите категорию --</option>
            <?php foreach ($categories as $cat): ?>
                <option value="<?= $cat['id'] ?>" <?= (isset($news->category_id)
&& $news->category_id == $cat['id']) ? 'selected' : '' ?>>
```

```
                    <?= htmlspecialchars($cat['name']) ?>
                </option>
            <?php endforeach; ?>
        </select>
    </div>

    <div class="form-group">
        <label for="content_html">Содержимое (HTML) *</label>
        <textarea name="content_html" id="content_html" rows="15" required><?=
htmlspecialchars($news->content_html ?? '') ?></textarea>
        <small>Можно использовать HTML теги</small>
    </div>

    <input type="hidden" name="content_json" value='{}'>

    <div class="form-group">
        <label for="status">Статус</label>
        <select name="status" id="status">
            <option value="draft" <?= (isset($news->status) && $news->status ===
'draft') ? 'selected' : '' ?>>Черновик</option>
            <option value="published" <?= (isset($news->status) && $news->status
=== 'published') ? 'selected' : '' ?>>Опубликовано</option>
        </select>
    </div>

    <div style="margin-top: 20px;">
        <button type="submit" class="btn">💾 Сохранить</button>
        <a href="/admin" class="btn" style="background: #6c757d;">← Отмена</a>
    </div>
</form>
app/src/Views/errors/404.php
<div style="text-align: center; padding: 50px;">
    <h1 style="font-size: 4em; color: #dc3545;">404</h1>
    <h2>Страница не найдена</h2>
    <p style="margin: 20px 0;">К сожалению, запрашиваемая страница не существует
или была удалена.</p>
    <div style="margin-top: 30px;">
        <a href="/" class="btn" style="margin-right: 10px;">🏠 На главную</a>
        <a href="/news" class="btn">📰 К новостям</a>
    </div>
</div>
```

Результат выполнения работы:

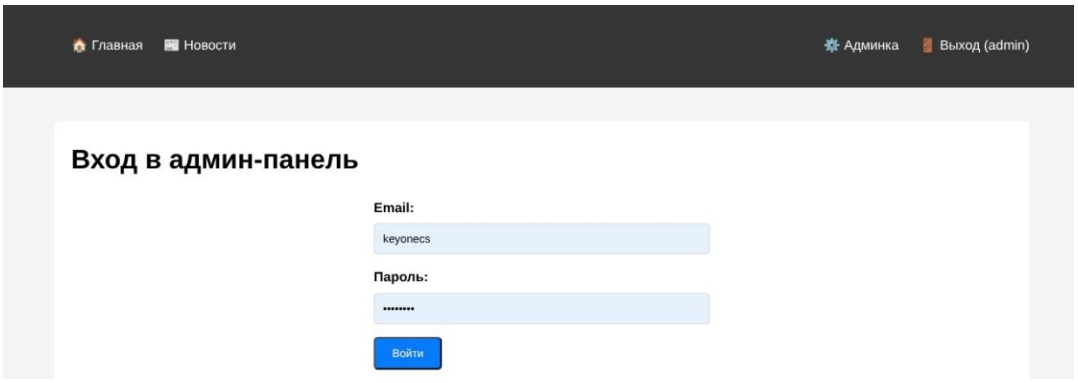


Рисунок 1 – Вход в панель администратора

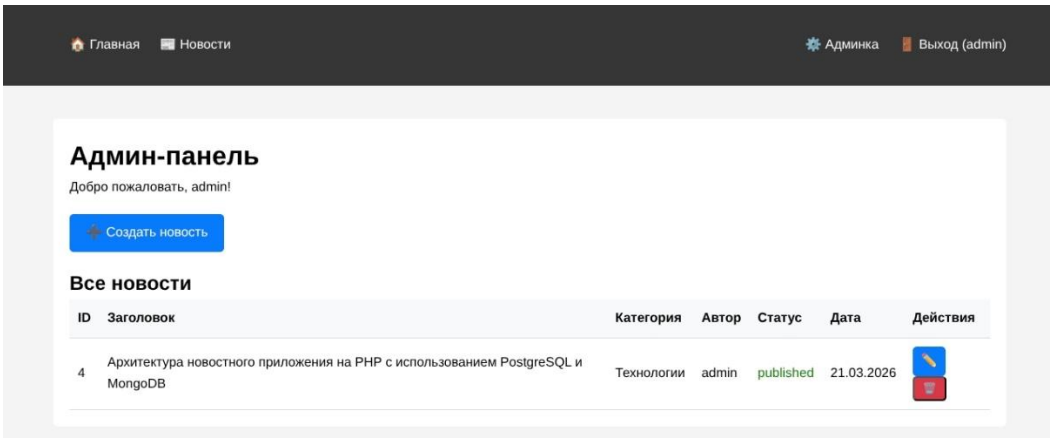


Рисунок 2 – Алмин-панель для создания новостей

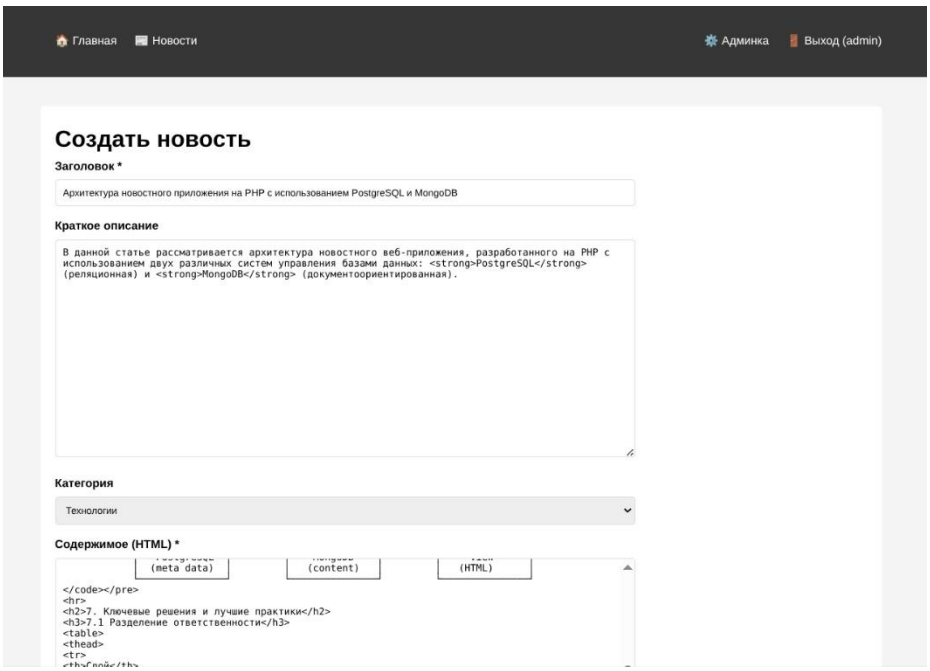


Рисунок 3 – Создание новости

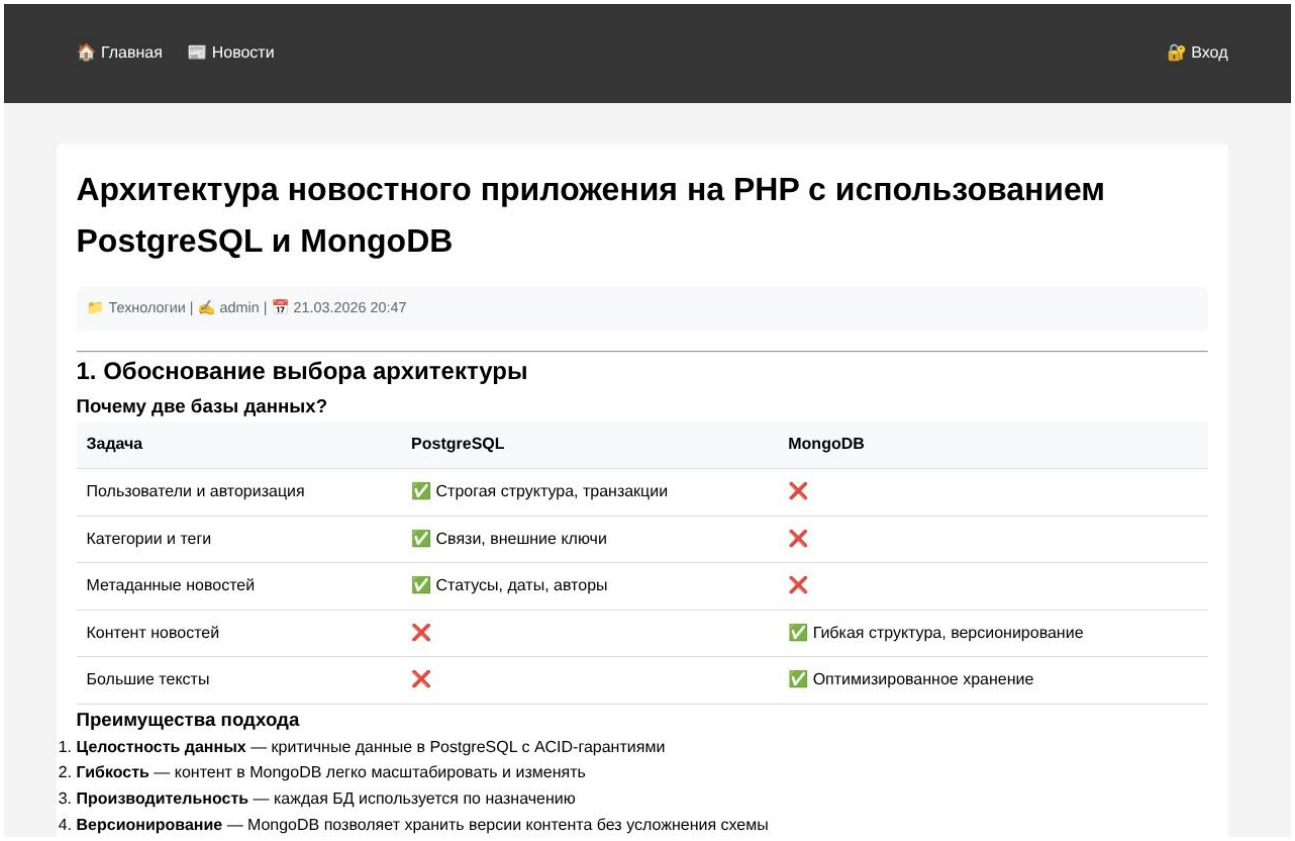


Рисунок 4 – Просмотр статьи

Вывод: в ходе выполнения лабораторной работы были сформированы практические навыки работы создание сервера, создающего запросы для базы данных.